

Java OOP Study Notes: Polymorphism

1. What is Polymorphism?

Polymorphism means "many forms." In Java, it allows objects to be treated as instances of their parent class while behaving according to their actual class.

Etymology

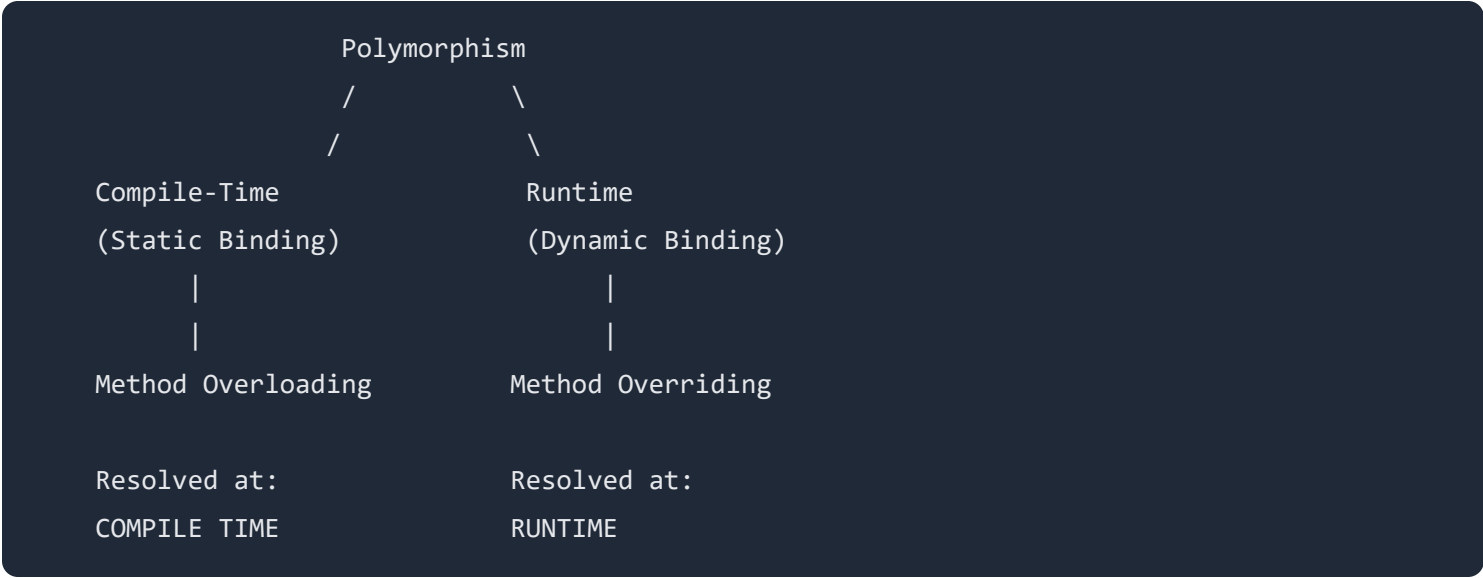
- **Poly** = Many
- **Morph** = Forms

Real-World Analogy

Action	Different Forms
<code>speak()</code>	Human speaks words, Dog barks, Cat meows
<code>draw()</code>	Circle draws circle, Rectangle draws rectangle
<code>calculate()</code>	<code>add(2,3)</code> , <code>add(2,3,4)</code> , <code>add(2.5, 3.5)</code>

Same action, different behaviors depending on the object or parameters.

2. Types of Polymorphism



Type	Mechanism	When Resolved	Also Called
Compile-time	Method Overloading	During compilation	Static Polymorphism, Early Binding
Runtime	Method Overriding	During execution	Dynamic Polymorphism, Late Binding

3. Compile-Time Polymorphism (Method Overloading)

Method Overloading = Multiple methods with the **same name** but **different parameters** in the same class.

The compiler decides which method to call based on the arguments passed.

Rules for Overloading

Must Differ	Can Be Same
Number of parameters	Method name
Type of parameters	Return type
Order of parameter types	Access modifier

Basic Example

```
class Calculator {

    // Method 1: Two integers
    int add(int a, int b) {
        System.out.println("add(int, int) called");
        return a + b;
    }

    // Method 2: Three integers (different number of params)
    int add(int a, int b, int c) {
        System.out.println("add(int, int, int) called");
        return a + b + c;
    }

    // Method 3: Two doubles (different type)
    double add(double a, double b) {
        System.out.println("add(double, double) called");
        return a + b;
    }

    // Method 4: int and double (different order)
    double add(int a, double b) {
        System.out.println("add(int, double) called");
        return a + b;
    }

    // Method 5: double and int (different order)
    double add(double a, int b) {
        System.out.println("add(double, int) called");
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();

        System.out.println(calc.add(5, 10));           // Calls Method 1
```

```

        System.out.println(calc.add(5, 10, 15));    // Calls Method 2
        System.out.println(calc.add(5.5, 10.5));    // Calls Method 3
        System.out.println(calc.add(5, 10.5));      // Calls Method 4
        System.out.println(calc.add(5.5, 10));      // Calls Method 5
    }
}

```

Output:

```

add(int, int) called
15
add(int, int, int) called
30
add(double, double) called
16.0
add(int, double) called
15.5
add(double, int) called
15.5

```

How Compiler Resolves Overloading

```

calc.add(5, 10);
  |
  ▼
Compiler checks all add() methods:
├─ add(int, int)      ← MATCH! ✓
├─ add(int, int, int) ← No (wrong param count)
├─ add(double, double) ← No (types don't match exactly)
├─ add(int, double)   ← No (second param type wrong)
└─ add(double, int)   ← No (first param type wrong)
  |
  ▼
Binds to add(int, int) at COMPILE TIME

```

Type Promotion in Overloading

If no exact match exists, Java **promotes** smaller types to larger types:

```
byte → short → int → long → float → double  
char → int
```

```
class Demo {  
    void show(int a) {  
        System.out.println("int: " + a);  
    }  
  
    void show(double a) {  
        System.out.println("double: " + a);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Demo d = new Demo();  
  
        d.show(10);           // int → calls show(int)  
        d.show(10.5);         // double → calls show(double)  
        d.show('A');          // char promoted to int → calls show(int)  
        d.show(10L);          // long promoted to double → calls show(double)  
    }  
}
```

Output:

```
int: 10  
double: 10.5  
int: 65  
double: 10.0
```

What Does NOT Count as Overloading

```
class Invalid {  
  
    // Original method  
    int calculate(int a, int b) {  
        return a + b;  
    }  
  
    // ERROR! Only return type is different  
    // double calculate(int a, int b) {  
    //     return a + b;  
    // }  
  
    // ERROR! Only access modifier is different  
    // public int calculate(int a, int b) {  
    //     return a + b;  
    // }  
  
    // ERROR! Only parameter names are different  
    // int calculate(int x, int y) {  
    //     return x + y;  
    // }  
}
```

Practical Overloading Examples

Constructor Overloading

```
class Student {
    String name;
    int age;
    String course;

    Student() {
        this("Unknown", 0, "Undeclared");
    }

    Student(String name) {
        this(name, 0, "Undeclared");
    }

    Student(String name, int age) {
        this(name, age, "Undeclared");
    }

    Student(String name, int age, String course) {
        this.name = name;
        this.age = age;
        this.course = course;
    }
}
```

println() - Classic Example

```
System.out.println();           // No args
System.out.println("Hello");    // String
System.out.println(42);         // int
System.out.println(3.14);       // double
System.out.println(true);       // boolean
System.out.println('A');        // char
System.out.println(new int[]{1,2,3}); // Object
```

The `println()` method is overloaded for different types!

4. Runtime Polymorphism (Method Overriding)

Method Overriding = Child class provides a specific implementation for a method already defined in its parent class.

The JVM decides which method to call based on the **actual object type** at runtime.

Rules for Overriding

Rule	Description
Same method name	Must match parent exactly
Same parameters	Same number, type, order
IS-A relationship	Must be in inheritance hierarchy
Same or covariant return	Can return subtype
Cannot reduce access	<code>protected</code> → <code>public</code> OK, <code>public</code> → <code>private</code> NOT OK
Cannot override final	Final methods cannot be overridden
Cannot override static	Static methods are hidden, not overridden

Basic Example

```
class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }

    void eat() {
        System.out.println("Animal eats food");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Dog barks: Woof! Woof!");
    }

    @Override
    void eat() {
        System.out.println("Dog eats meat and kibble");
    }
}

class Cat extends Animal {
    @Override
    void makeSound() {
        System.out.println("Cat meows: Meow!");
    }

    @Override
    void eat() {
        System.out.println("Cat eats fish and cat food");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a1 = new Animal();
        Animal a2 = new Dog();    // Parent reference, child object
    }
}
```

```
Animal a3 = new Cat();    // Parent reference, child object

a1.makeSound(); // Animal makes a sound
a2.makeSound(); // Dog barks: Woof! Woof!
a3.makeSound(); // Cat meows: Meow!
}
}
```

The Power of Runtime Polymorphism

```
public class Zoo {
    public static void main(String[] args) {

        // Array of parent type holding different child objects
        Animal[] animals = {
            new Dog(),
            new Cat(),
            new Dog(),
            new Cat(),
            new Animal()
        };

        // Same code handles ALL types!
        for (Animal animal : animals) {
            animal.makeSound(); // Correct method called for each
        }
    }
}
```

Output:

```
Dog barks: Woof! Woof!  
Cat meows: Meow!  
Dog barks: Woof! Woof!  
Cat meows: Meow!  
Animal makes a sound
```

Without polymorphism, you'd need separate code for each animal type!

5. Dynamic Method Dispatch

Dynamic Method Dispatch is the mechanism by which a call to an overridden method is resolved at **runtime** rather than compile-time.

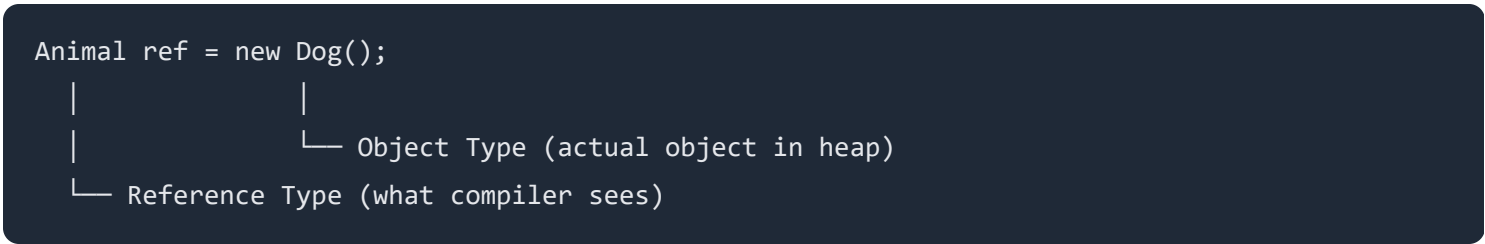
This is the technical implementation of runtime polymorphism.

How It Works

```
Animal ref = new Dog(); // Parent reference, Child object  
ref.makeSound();         // Which makeSound() is called?
```



Key Concept: Reference Type vs Object Type



Aspect	Determined By
Which methods can be called	Reference type (compile-time)
Which version of method runs	Object type (runtime)

```
Animal ref = new Dog();

ref.makeSound(); // ✓ OK - Animal has makeSound()
// ref.bark();   // ✗ ERROR - Animal doesn't have bark()!

// Even though the object IS a Dog, compiler only sees Animal reference
```

Complete Dynamic Dispatch Example

```
class Shape {
    void draw() {
        System.out.println("Drawing a shape");
    }

    double getArea() {
        return 0;
    }
}

class Circle extends Shape {
    double radius;

    Circle(double radius) {
        this.radius = radius;
    }

    @Override
    void draw() {
        System.out.println("Drawing a circle with radius " + radius);
    }

    @Override
    double getArea() {
        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape {
    double width, height;

    Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    @Override
    void draw() {
```

```
        System.out.println("Drawing rectangle " + width + "x" + height);
    }

    @Override
    double getArea() {
        return width * height;
    }
}

class Triangle extends Shape {
    double base, height;

    Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }

    @Override
    void draw() {
        System.out.println("Drawing triangle with base " + base);
    }

    @Override
    double getArea() {
        return 0.5 * base * height;
    }
}

public class DrawingApp {

    // This method works with ANY shape!
    static void renderShape(Shape shape) {
        shape.draw();
        System.out.println("Area: " + shape.getArea());
        System.out.println();
    }

    public static void main(String[] args) {
        Shape[] shapes = {
            new Circle(5),
```

```

        new Rectangle(4, 6),
        new Triangle(3, 8),
        new Circle(10)
    };

    System.out.println("=== Rendering All Shapes ===\n");

    for (Shape shape : shapes) {
        renderShape(shape); // Dynamic dispatch in action!
    }

    // Calculate total area
    double totalArea = 0;
    for (Shape shape : shapes) {
        totalArea += shape.getArea();
    }
    System.out.println("Total Area: " + totalArea);
}
}

```

Output:

```

=== Rendering All Shapes ===

```

```

Drawing a circle with radius 5.0
Area: 78.53981633974483

```

```

Drawing rectangle 4.0x6.0
Area: 24.0

```

```

Drawing triangle with base 3.0
Area: 12.0

```

```

Drawing a circle with radius 10.0
Area: 314.1592653589793

```

```

Total Area: 428.6990816987241

```


Virtual Method Table (VTable)

Under the hood, JVM uses a **Virtual Method Table** for dynamic dispatch:

```
Each class has a VTable with method addresses
```

Animal VTable

makeSound	→ 0x100
eat	→ 0x104

Dog VTable

makeSound	→ 0x200	← Dog's version
eat	→ 0x204	← Dog's version

```
When: Animal ref = new Dog();  
      ref.makeSound();
```

```
JVM: 1. Look at actual object → Dog  
      2. Find Dog's VTable  
      3. Get makeSound address → 0x200  
      4. Execute Dog.makeSound()
```

6. The instanceof Keyword

The `instanceof` keyword checks whether an object is an instance of a specific class or interface.

Syntax

```
object instanceof ClassName // Returns true or false
```

Basic Usage

```
class Animal { }
class Dog extends Animal { }
class Cat extends Animal { }

public class Main {
    public static void main(String[] args) {
        Animal animal = new Dog();
        Dog dog = new Dog();

        System.out.println(dog instanceof Dog);    // true
        System.out.println(dog instanceof Animal); // true (Dog IS-A Animal)
        System.out.println(animal instanceof Dog); // true (actual object is Dog)
        System.out.println(animal instanceof Cat); // false
        System.out.println(animal instanceof Animal); // true

        // null check
        Dog nullDog = null;
        System.out.println(nullDog instanceof Dog); // false (null is never instanceof)
    }
}
```

Why Use instanceof ?

1. Safe Downcasting

```
class Animal {
    void eat() { System.out.println("Eating"); }
}

class Dog extends Animal {
    void bark() { System.out.println("Barking!"); }
}

class Cat extends Animal {
    void meow() { System.out.println("Meowing!"); }
}

public class Main {
    public static void main(String[] args) {
        Animal[] animals = { new Dog(), new Cat(), new Dog() };

        for (Animal animal : animals) {
            animal.eat(); // OK - all animals can eat

            // Want to call bark() only for dogs
            // animal.bark(); // ERROR! Animal doesn't have bark()

            // Solution: Check type first, then cast
            if (animal instanceof Dog) {
                Dog dog = (Dog) animal; // Safe downcast
                dog.bark();
            }

            if (animal instanceof Cat) {
                Cat cat = (Cat) animal; // Safe downcast
                cat.meow();
            }

            System.out.println();
        }
    }
}
```

```
}  
}
```

Output:

```
Eating  
Barking!  
  
Eating  
Meowing!  
  
Eating  
Barking!
```

Without `instanceof` Check (Dangerous!)

```
Animal animal = new Cat();  
Dog dog = (Dog) animal; // ClassCastException at runtime!  
dog.bark();
```

Pattern Matching with `instanceof` (Java 16+)

Modern Java combines type check and cast in one step:

```
// Old way (pre-Java 16)
if (animal instanceof Dog) {
    Dog dog = (Dog) animal;
    dog.bark();
}

// New way (Java 16+) - Pattern Matching
if (animal instanceof Dog dog) {
    dog.bark(); // dog is already cast and available!
}

// Works with negation too
if (!(animal instanceof Dog dog)) {
    return;
}
dog.bark(); // dog is available here
```

instanceof with Interfaces

```
interface Swimmable {
    void swim();
}

interface Flyable {
    void fly();
}

class Duck implements Swimmable, Flyable {
    public void swim() { System.out.println("Duck swimming"); }
    public void fly() { System.out.println("Duck flying"); }
}

class Fish implements Swimmable {
    public void swim() { System.out.println("Fish swimming"); }
}

public class Main {
    public static void main(String[] args) {
        Object[] creatures = { new Duck(), new Fish(), "Not an animal" };

        for (Object obj : creatures) {
            System.out.println("Checking: " + obj.getClass().getSimpleName());

            if (obj instanceof Swimmable) {
                System.out.println("  Can swim!");
                ((Swimmable) obj).swim();
            }

            if (obj instanceof Flyable) {
                System.out.println("  Can fly!");
                ((Flyable) obj).fly();
            }

            System.out.println();
        }
    }
}
```

```
}  
}
```

Output:

```
Checking: Duck  
    Can swim!  
Duck swimming  
    Can fly!  
Duck flying  
  
Checking: Fish  
    Can swim!  
Fish swimming  
  
Checking: String
```

`instanceof` Hierarchy Check

```
class A { }  
class B extends A { }  
class C extends B { }  
  
public class Main {  
    public static void main(String[] args) {  
        C obj = new C();  
  
        System.out.println(obj instanceof C); // true  
        System.out.println(obj instanceof B); // true (C extends B)  
        System.out.println(obj instanceof A); // true (C extends B extends A)  
        System.out.println(obj instanceof Object); // true (everything extends Object)  
    }  
}
```

Inheritance Chain

Object

|

A obj instanceof A? ✓

|

B obj instanceof B? ✓

|

C ← obj obj instanceof C? ✓



7. Complete Polymorphism Example

```
// =====  
// INTERFACES  
// =====  
interface Taxable {  
    double calculateTax();  
}  
  
interface Discountable {  
    double applyDiscount(double percentage);  
}  
  
// =====  
// BASE CLASS  
// =====  
abstract class Product {  
    protected String name;  
    protected double price;  
  
    Product(String name, double price) {  
        this.name = name;  
        this.price = price;  
    }  
  
    // Method to be overridden  
    abstract String getCategory();  
  
    // Common method  
    void displayInfo() {  
        System.out.println("Product: " + name);  
        System.out.println("Price: $" + price);  
        System.out.println("Category: " + getCategory());  
    }  
  
    // Overloaded method - compile-time polymorphism  
    double calculateTotal() {  
        return price;  
    }  
}
```

```

double calculateTotal(int quantity) {
    return price * quantity;
}

double calculateTotal(int quantity, double taxRate) {
    return price * quantity * (1 + taxRate);
}
}

// =====
// ELECTRONICS - Taxable
// =====

class Electronics extends Product implements Taxable {
    private String brand;
    private int warrantyYears;

    Electronics(String name, double price, String brand, int warrantyYears) {
        super(name, price);
        this.brand = brand;
        this.warrantyYears = warrantyYears;
    }

    @Override
    String getCategory() {
        return "Electronics";
    }

    @Override
    public double calculateTax() {
        return price * 0.18; // 18% tax on electronics
    }

    @Override
    void displayInfo() {
        super.displayInfo();
        System.out.println("Brand: " + brand);
        System.out.println("Warranty: " + warrantyYears + " years");
        System.out.println("Tax: $" + calculateTax());
    }
}

```

```
// =====  
// CLOTHING - Discountable  
// =====  
class Clothing extends Product implements Discountable {  
    private String size;  
    private String color;  
  
    Clothing(String name, double price, String size, String color) {  
        super(name, price);  
        this.size = size;  
        this.color = color;  
    }  
  
    @Override  
    String getCategory() {  
        return "Clothing";  
    }  
  
    @Override  
    public double applyDiscount(double percentage) {  
        return price * (1 - percentage / 100);  
    }  
  
    @Override  
    void displayInfo() {  
        super.displayInfo();  
        System.out.println("Size: " + size);  
        System.out.println("Color: " + color);  
    }  
}  
  
// =====  
// GROCERY - Taxable & Discountable  
// =====  
class Grocery extends Product implements Taxable, Discountable {  
    private String expiryDate;  
    private boolean organic;  
  
    Grocery(String name, double price, String expiryDate, boolean organic) {
```

```

        super(name, price);
        this.expiryDate = expiryDate;
        this.organic = organic;
    }

    @Override
    String getCategory() {
        return organic ? "Organic Grocery" : "Grocery";
    }

    @Override
    public double calculateTax() {
        return organic ? price * 0.05 : price * 0.08; // Lower tax for organic
    }

    @Override
    public double applyDiscount(double percentage) {
        // Max 20% discount on groceries
        double actualDiscount = Math.min(percentage, 20);
        return price * (1 - actualDiscount / 100);
    }

    @Override
    void displayInfo() {
        super.displayInfo();
        System.out.println("Expiry: " + expiryDate);
        System.out.println("Organic: " + (organic ? "Yes" : "No"));
        System.out.println("Tax: $" + calculateTax());
    }
}

// =====
// SHOPPING CART - Uses Polymorphism
// =====

class ShoppingCart {
    private Product[] items;
    private int count;

    ShoppingCart(int capacity) {
        items = new Product[capacity];
    }
}

```

```

        count = 0;
    }

    void addItem(Product product) {
        if (count < items.length) {
            items[count++] = product;
            System.out.println("Added: " + product.name);
        }
    }

    void displayAllItems() {
        System.out.println("\n=====");
        System.out.println("          SHOPPING CART CONTENTS          ");
        System.out.println("=====\\n");

        for (int i = 0; i < count; i++) {
            System.out.println("--- Item " + (i + 1) + " ---");
            items[i].displayInfo(); // Runtime polymorphism!
            System.out.println();
        }
    }

    double calculateTotalTax() {
        double totalTax = 0;

        for (int i = 0; i < count; i++) {
            // Use instanceof to check if item is taxable
            if (items[i] instanceof Taxable) {
                Taxable taxableItem = (Taxable) items[i];
                totalTax += taxableItem.calculateTax();
            }
        }

        return totalTax;
    }

    void applyDiscountToClothing(double percentage) {
        System.out.println("\nApplying " + percentage + "% discount to clothing:");

        for (int i = 0; i < count; i++) {

```

```

        if (items[i] instanceof Clothing) {
            Clothing clothing = (Clothing) items[i];
            double newPrice = clothing.applyDiscount(percentage);
            System.out.println("  " + clothing.name + ": $" + clothing.price +
                               " → $" + newPrice);
        }
    }
}

double getGrandTotal() {
    double subtotal = 0;
    double tax = 0;

    for (int i = 0; i < count; i++) {
        subtotal += items[i].price;

        if (items[i] instanceof Taxable) {
            tax += ((Taxable) items[i]).calculateTax();
        }
    }

    return subtotal + tax;
}

void printReceipt() {
    System.out.println("\n=====");
    System.out.println("                RECEIPT                ");
    System.out.println("=====");

    double subtotal = 0;
    double totalTax = 0;

    for (int i = 0; i < count; i++) {
        Product p = items[i];
        System.out.printf("%-20s $%.2f\n", p.name, p.price);
        subtotal += p.price;

        if (p instanceof Taxable) {
            double tax = ((Taxable) p).calculateTax();
            totalTax += tax;
        }
    }
}

```

```

        System.out.printf("    (tax: $%.2f)%n", tax);
    }
}

System.out.println("=====");
System.out.printf("Subtotal:          $%.2f%n", subtotal);
System.out.printf("Total Tax:          $%.2f%n", totalTax);
System.out.println("=====");
System.out.printf("GRAND TOTAL:          $%.2f%n", subtotal + totalTax);
System.out.println("=====");
}
}

// =====
// MAIN - Testing All Polymorphism Features
// =====

public class Main {
    public static void main(String[] args) {

        // Create products
        Electronics laptop = new Electronics("MacBook Pro", 1999.99, "Apple", 2);
        Electronics phone = new Electronics("Galaxy S24", 899.99, "Samsung", 1);
        Clothing shirt = new Clothing("Cotton T-Shirt", 29.99, "M", "Blue");
        Clothing jeans = new Clothing("Slim Fit Jeans", 59.99, "32", "Black");
        Grocery apples = new Grocery("Organic Apples", 5.99, "2025-03-01", true);
        Grocery bread = new Grocery("Whole Wheat Bread", 3.49, "2025-02-25", false);

        // Create cart and add items
        ShoppingCart cart = new ShoppingCart(10);
        cart.addItem(laptop);
        cart.addItem(phone);
        cart.addItem(shirt);
        cart.addItem(jeans);
        cart.addItem(apples);
        cart.addItem(bread);

        // Display all items (runtime polymorphism)
        cart.displayAllItems();

        // Apply discount to clothing (instanceof)
    }
}

```

```
        cart.applyDiscountToClothing(25);

        // Print receipt (polymorphism + instanceof)
        cart.printReceipt();

        // =====
        // Demonstrate compile-time polymorphism (overloading)
        // =====
        System.out.println("\n==== Method Overloading Demo ===");
        System.out.println("laptop.calculateTotal() = $" + laptop.calculateTotal());
        System.out.println("laptop.calculateTotal(2) = $" + laptop.calculateTotal(2));
        System.out.println("laptop.calculateTotal(2, 0.1) = $" + laptop.calculateTotal(2, 0.1)
    }
}
```

Output:

Added: MacBook Pro
Added: Galaxy S24
Added: Cotton T-Shirt
Added: Slim Fit Jeans
Added: Organic Apples
Added: Whole Wheat Bread

SHOPPING CART CONTENTS

--- Item 1 ---
Product: MacBook Pro
Price: \$1999.99
Category: Electronics
Brand: Apple
Warranty: 2 years
Tax: \$359.9982

--- Item 2 ---
Product: Galaxy S24
Price: \$899.99
Category: Electronics
Brand: Samsung
Warranty: 1 years
Tax: \$161.9982

... (more items)

Applying 25% discount to clothing:
Cotton T-Shirt: \$29.99 → \$22.4925
Slim Fit Jeans: \$59.99 → \$44.9925

RECEIPT

MacBook Pro	\$1999.99
(tax: \$359.9982)	
Galaxy S24	\$899.99
(tax: \$161.9982)	

Cotton T-Shirt\$29.99

Slim Fit Jeans\$59.99

Organic Apples\$5.99

(tax: \$0.2995)

Whole Wheat Bread\$3.49

(tax: \$0.2792)

Subtotal:\$2999.44

Total Tax:\$522.57

GRAND TOTAL:\$3522.01

Method Overloading Demo

laptop.calculateTotal() = \$1999.99

laptop.calculateTotal(2) = \$3999.98

laptop.calculateTotal(2, 0.1) = \$4399.978

8. Comparison Table

Aspect	Compile-Time (Overloading)	Runtime (Overriding)
Binding	Static (compile-time)	Dynamic (runtime)
Methods	Same name, different params	Same name, same params
Classes	Same class	Parent-child classes
Inheritance	Not required	Required
Return Type	Can differ	Must be same/covariant
Access Modifier	Can differ	Cannot be more restrictive
Performance	Faster (resolved early)	Slightly slower (lookup)
Flexibility	Less flexible	More flexible

9. Key Points to Remember

1. **Polymorphism** = "Many forms" — same action, different behaviors
 2. **Compile-time** = Method Overloading (same name, different params)
 3. **Runtime** = Method Overriding (same signature in child class)
 4. **Dynamic Method Dispatch** = JVM decides method at runtime based on actual object
 5. **Reference type** determines what methods you CAN call
 6. **Object type** determines which VERSION runs
 7. **instanceof** checks if object is instance of a class/interface
 8. **Always use instanceof** before downcasting to avoid ClassCastException
 9. **@Override** annotation helps catch errors at compile time
 10. **Overloading** is resolved by compiler, **Overriding** is resolved by JVM
- ```
'; triggerDownload(docTitle.replace(/^[^a-zA-Z0-9\s-]/g, '') + '.html', html, 'text/html;charset=utf-8'); }
function downloadPDF() { document.getElementById('downloadDropdown').classList.remove('show'); //
Force light mode for PDF (dark backgrounds print poorly) const wasDark =
document.documentElement.classList.contains('dark'); if (wasDark)
document.documentElement.classList.remove('dark'); const element =
document.getElementById('document-content'); const opt = { margin: [0.75, 0.75, 0.75, 0.75], filename:
docTitle.replace(/^[^a-zA-Z0-9\s-]/g, '') + '.pdf', image: { type: 'jpeg', quality: 0.98 }, html2canvas: { scale:
1.5, useCORS: true, backgroundColor: '#ffffff' }, jsPDF: { unit: 'in', format: 'letter', orientation: 'portrait' },
pagebreak: { mode: ['css', 'legacy'] } }; html2pdf().set(opt).from(element).save().then(function() { if
(wasDark) document.documentElement.classList.add('dark'); }); } /* Syntax highlighting and mermaid
rendering */ (async function init() { const isDark = window.matchMedia('(prefers-color-scheme:
dark)').matches; // Highlight code blocks (skip mermaid) document.querySelectorAll('pre
code').forEach(function(block) { const lang = block.className.replace('language-', ''); if (lang !==
'mermaid') { hljs.highlightElement(block); } }); updateHljsTheme(isDark); // Render mermaid diagrams try {
mermaid.initialize({ startOnLoad: false, theme: isDark ? 'dark' : 'default', securityLevel: 'loose' }); const
mermaidBlocks = document.querySelectorAll('pre code.language-mermaid'); for (let i = 0; i <
mermaidBlocks.length; i++) { const block = mermaidBlocks[i]; const pre = block.parentElement; const
code = block.textContent; try { const { svg } = await mermaid.render('mermaid-' + i, code); const
container = document.createElement('div'); container.className = 'mermaid-container';
container.innerHTML = svg; pre.replaceWith(container); } catch (err) { console.warn('Mermaid render
```

```
failed:', err); } } } catch (err) { console.warn('Mermaid init failed:', err); } })(); function
updateHljsTheme(isDark) { document.getElementById('hljs-dark').disabled = !isDark;
document.getElementById('hljs-light').disabled = isDark; } /* Dark mode sync from parent */
window.addEventListener('message', function(e) { if (e.data && e.data.type === 'darkMode') { const
isDark = e.data.dark; document.documentElement.classList.toggle('dark', isDark);
updateHljsTheme(isDark); // Re-init mermaid theme try { mermaid.initialize({ startOnLoad: false, theme:
isDark ? 'dark' : 'default', securityLevel: 'loose' }); } catch (err) { } } });
```